NEC
Empowered by Innovation

University of Waterloo

Software Testing and Quality Assurance

Theory and Practice

Chapter 15

Software Reliability

© Naik & Tripathy

Outline of the Chapter

- What is Reliability?
- Definitions of Software Reliability
- Factors Influencing Software Reliability
- Applications of Software Reliability
- Operational Profiles
- Reliability Models
- Summary

© Naik & Tripathy

What is Reliability?

- Reliability is a broad concept.
  - It is applied whenever we expect something to behave in a certain way.
- Reliability is one of the metrics that are used to measure quality.
- It is a user-oriented quality factor relating to system operation.
  - Intuitively, if the users of a system rarely experience failure, the system is considered to be more reliable than one that fails more often.
- A system without faults is considered to be highly reliable.
  - Constructing a correct system is a difficult task.
  - Even an incorrect system may be considered to be reliable if the frequency of failure is "acceptable."
- Key concepts in discussing reliability:
  - Fault
  - Failure
  - Time
  - Three kinds of time intervals: MTTR, MTTF, MTBF

What is Reliability?

- Failure
  - A failure is said to occur if the **observable** outcome of a **program execution** is different from the expected outcome.
- Fault
  - The adjudged cause of failure is called a fault.
  - Example: A failure may be cause by a defective block of code.
- Time
  - Time is a key concept in the formulation of reliability. If the time gap between two successive failures is short, we say that the system is less reliable.
  - Two forms of time are considered.
    - Execution time (τ)
    - Calendar time (t)

What is Reliability?

- MTTF: Mean Time To Failure
- MTTR: Mean Time To Repair
- MTBF: Mean Time Between Failures (= MTTF + MTTR)

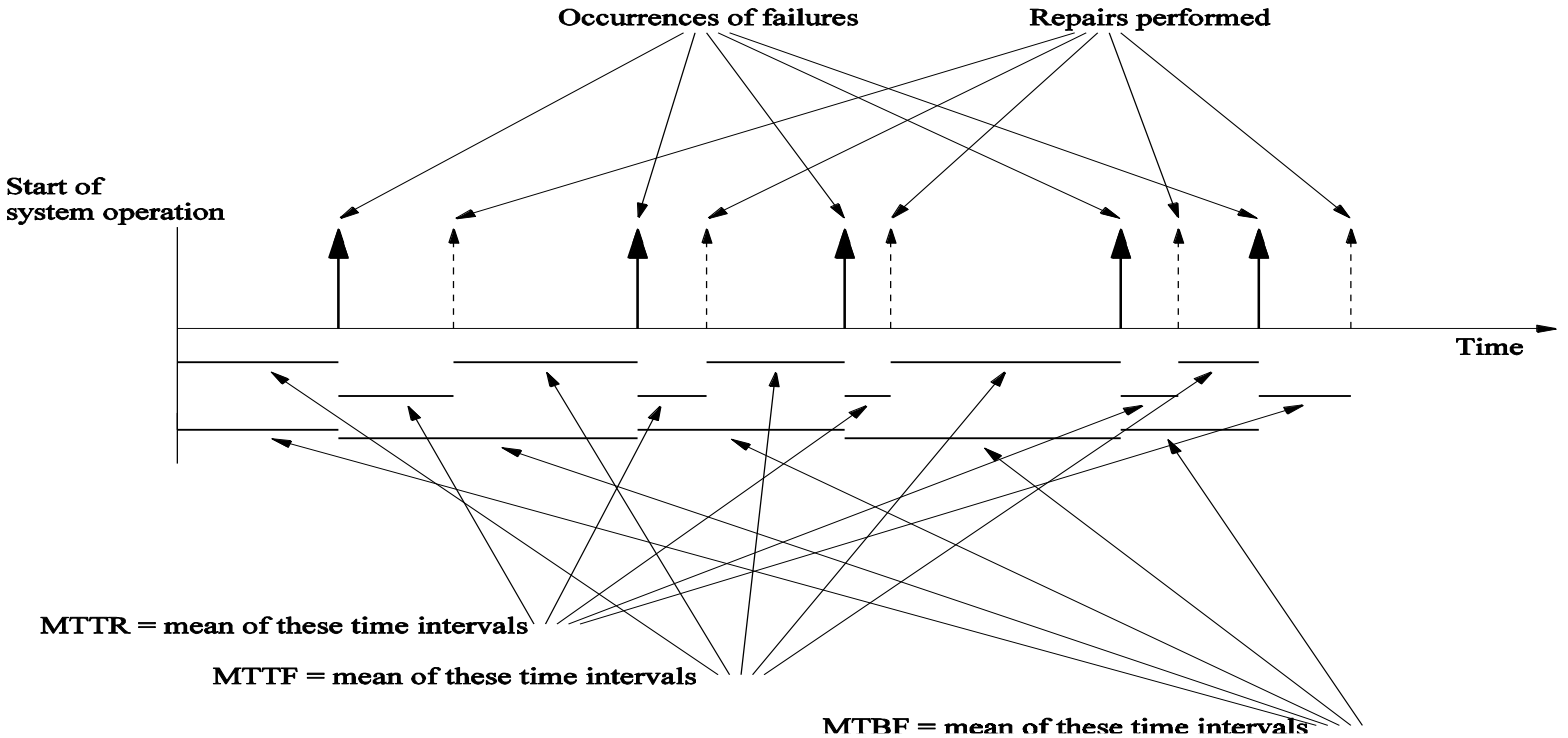


Figure 15.1: Relationship between MTTR, MTTF, and MTBF.

© Naik & Tripathy

What is Reliability?

- Two ways to measure reliability
  - Counting failures in periodic intervals
    - Observer the trend of **cumulative failure count - $\mu(\tau)$.**
  - Failure intensity
    - Observe the trend of **number of failures per unit time – $\lambda(\tau)$.**
- $\mu(\tau)$
  - This denotes the **total number of failures** observed until execution time τ from the beginning of system execution.
- $\lambda(\tau)$
  - This denotes the **number of failures observed per unit time** after τ time units of executing the system from the beginning. This is also called the failure intensity at time τ.
- Relationship between $\lambda(\tau)$ and $\mu(\tau)$
  - $\lambda(\tau) = d\mu(\tau)/d\tau$

Definitions of Software Reliability

- First definition
  - Software reliability is defined as the probability of failure-free operation of a software system for a specified time in a specified environment.
    - Key elements of the above definition
      - Probability of failure-free operation
      - Length of time of failure-free operation
      - A given execution environment
    - Example
      - The probability that a PC in a store is up and running for eight hours without crash is 0.99.
- Second definition
  - Failure intensity is a measure of the reliability of a software system operating in a given environment.
    - Example: An air traffic control system fails once in two years.
- Comparing the two
  - The first puts emphasis on MTTF, whereas the second on count.

© Naik & Tripathy

Factors Influencing Software Reliability

- A user's perception of the reliability of a software depends upon two categories of information.
  - The number of faults present in the software.
  - The ways users operate the system.
    - This is known as the *operational profile*.
- The fault count in a system is influenced by the following.
  - Size and complexity of code
  - Characteristics of the development process used
  - Education, experience, and training of development personnel
  - Operational environment

© Naik & Tripathy

Applications of Software Reliability

- Comparison of software engineering technologies
  - What is the cost of adopting a technology?
  - What is the return from the technology -- in terms of cost and quality?
- Measuring the progress of system testing
  - Key question: How of testing has been done?
  - The failure intensity measure tells us about the present quality of the system: high intensity means more tests are to be performed.
- Controlling the system in operation
  - The amount of change to a software for maintenance affects its reliability. Thus the amount of change to be effected in one go is determined by how much reliability we are ready to potentially lose.
- Better insight into software development processes
  - Quantification of quality gives us a better insight into the development processes.

© Naik & Tripathy

Operational Profiles

- Developed at AT&T Bell Labs.
- An OP describes how actual users operate a system.
  - An OP is a quantitative characterization of how a system will be used.
- Two ways to represent operational profiles
  - Tabular
  - Graphical

Operation	Operations per hour	Probability
Book checked out	450	0.45
Book returned in time	324	0.324
Book renewed	81	0.081
Book returned late	36	0.036
Book reported lost	9	0.009
...	...	...
Total	1000	1.0

Table 15.1: An example of operational profile of a library information system.

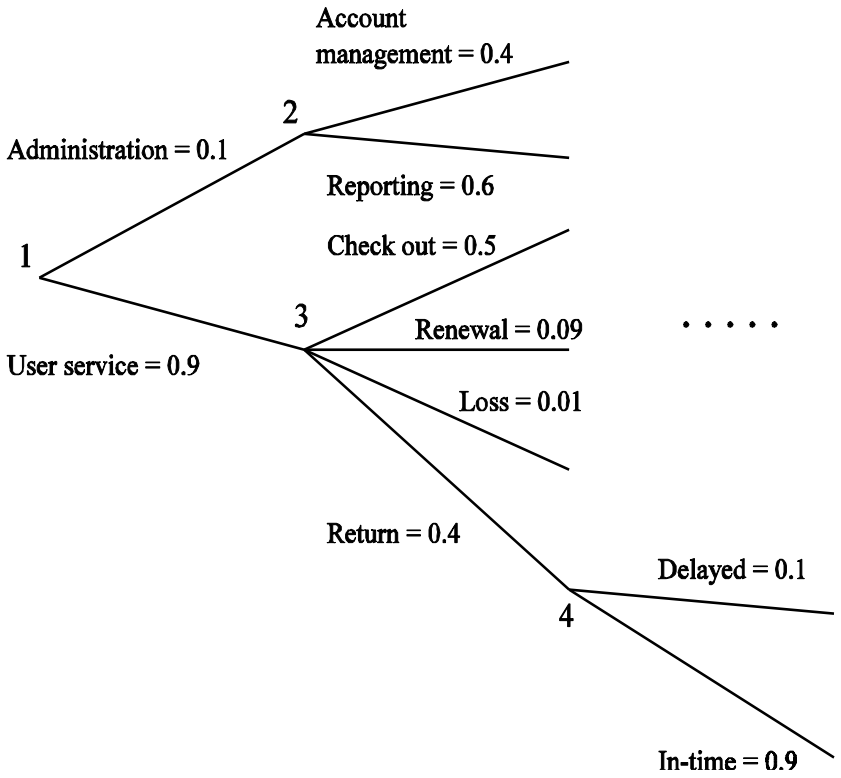


Figure 15.2: Graphical representation of operational profile of a library information system.

© Naik & Tripathy

Operational Profiles

- Use of operational profiles
  - For accurate estimation of the reliability of a system, test the system in the same way it will be actually used in the field.
- Other uses of operational profiles
  - Use an OP as a guiding document in designing user interfaces.
    - The more frequently used operations should be easy to use.
  - Use an OP to design an early version of a software for release.
    - This contains the more frequently used operations.
  - Use an OP to determine where to put more resources.

© Naik & Tripathy

Reliability Models

- Main idea
  - We develop mathematical models for $\lambda(\tau)$ and $\mu(\tau)$.

- Basic assumptions in developing a reliability model
  - Faults in the program are independent.
  - Execution time between failures is large w.r.t. instruction execution time.
  - Potential test space covers its use space.
  - The set of inputs per test run is randomly chosen.
  - The fault causing a failure is immediately fixed or else its re-occurrence is not counted again.

© Naik & Tripathy

Reliability Models

- Intuitive idea
  - As we observe another system failure and the corresponding fault is fixed, there will be fewer number of faults remaining in the system and the failure intensity will be smaller with each fault fixed.
  - In other words, as the cumulative failure count increases, **the failure intensity decreases.**
- Two decrement processes
  - Decrement process 1
    - The decrease in failure intensity after observing a failure and fixing the corresponding fault is **constant**.
      - This gives us the Basic model.
  - Decrement process 2
    - The decrease in failure intensity after observing a failure and fixing the corresponding fault is **smaller** than the previous decrease.
      - This gives us the Logarithmic model.

Reliability Models

- Parameters of the models
  - λ_0: The initial failure intensity observed at the beginning of system testing.
  - ν_0: The total number of system failures that we expect to observe over infinite time starting from the beginning of system testing.
  - θ: A parameter representing n0n-linear drop in failure intensity in the Logarithmic model.

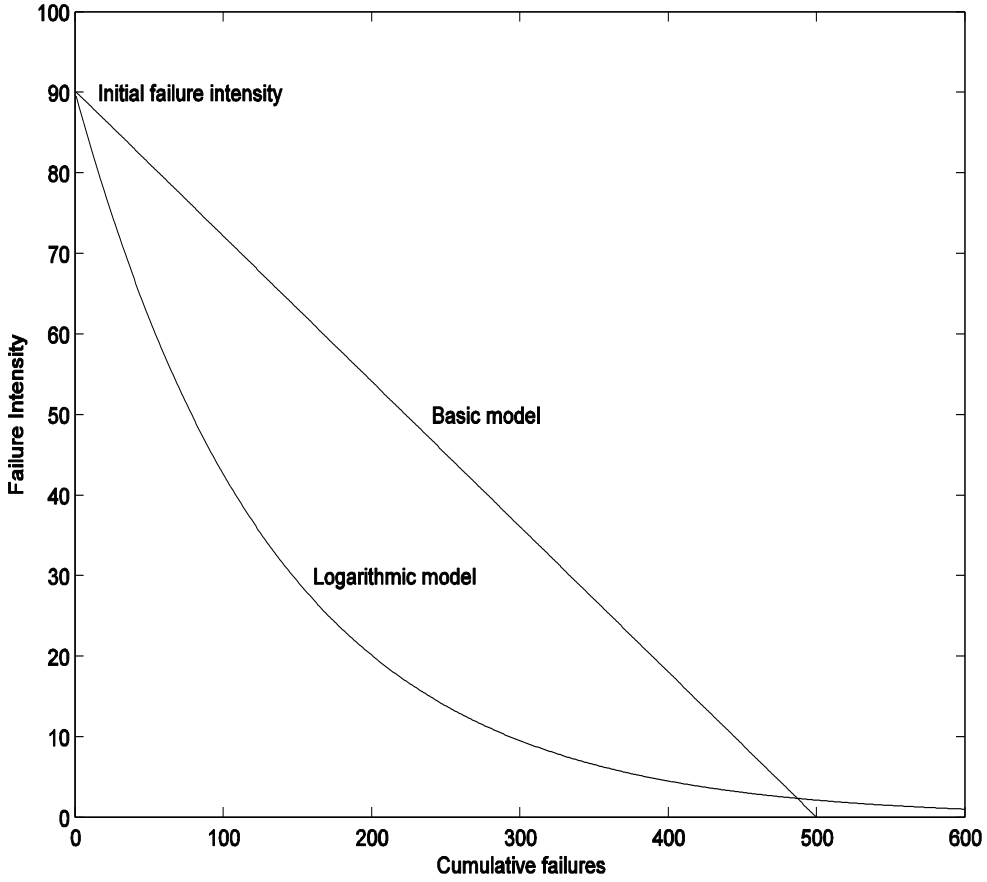


Figure 15.3: Failure intensity λ as a function of cumulative failures μ.

© Naik & Tripathy

Reliability Models

- Basic model

  Assumption: $\lambda(\mu) = \lambda_0 (1 - \mu/\nu_0)$

  $d\mu(\tau)/d\tau = \lambda_0 (1 - \mu(\tau)/\nu_0)$

  $\mu(\tau) = \lambda_0 (1 - \mu/\nu_0)$

  $\lambda(\tau) = \lambda_0.e^{-\lambda 0\, \tau/\nu 0}$

- Logarithmic model

  Assumption: $\lambda(\mu) = \lambda_0 e^{-\theta\mu}$

  $d\mu(\tau)/d\tau = \lambda_0 e^{-\theta\mu(\tau)}$

  $\mu(\tau) = \ln(\lambda_0\theta\tau + 1)/\theta$

  $\lambda(\tau) = \lambda_0/(\lambda_0\theta\tau + 1)$

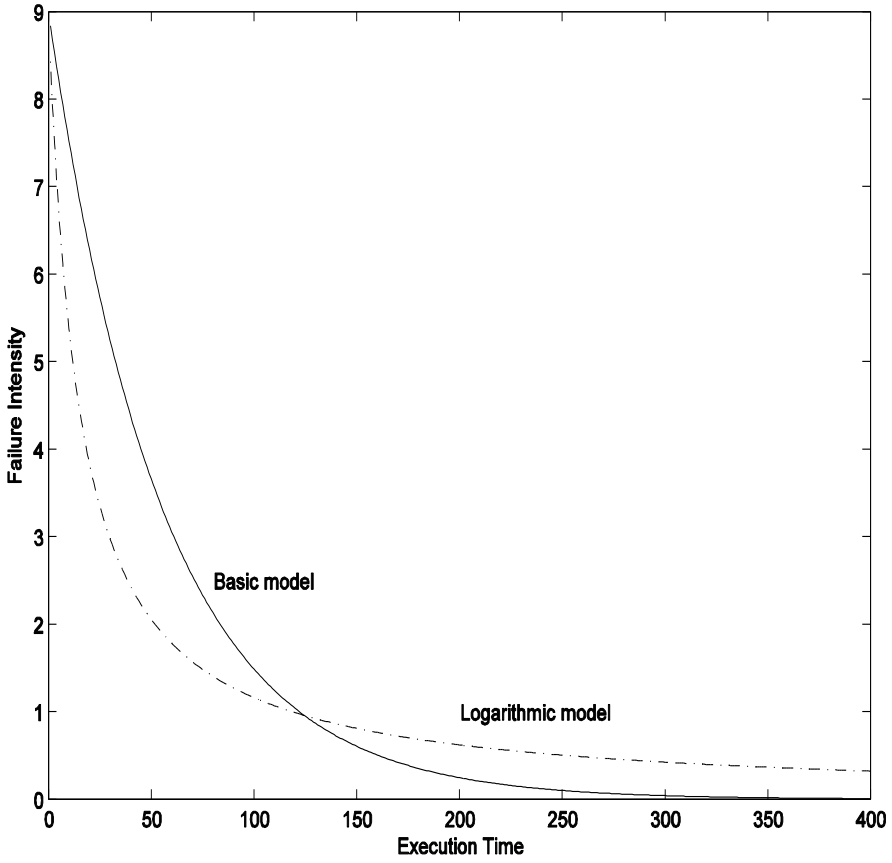


Figure 15.4: Failure intensity λ as a function of execution time τ ($\lambda_0 = 9$ failures/unit time, $\nu_0 = 500$ failures, $\theta = 0.0075$).

Reliability Models

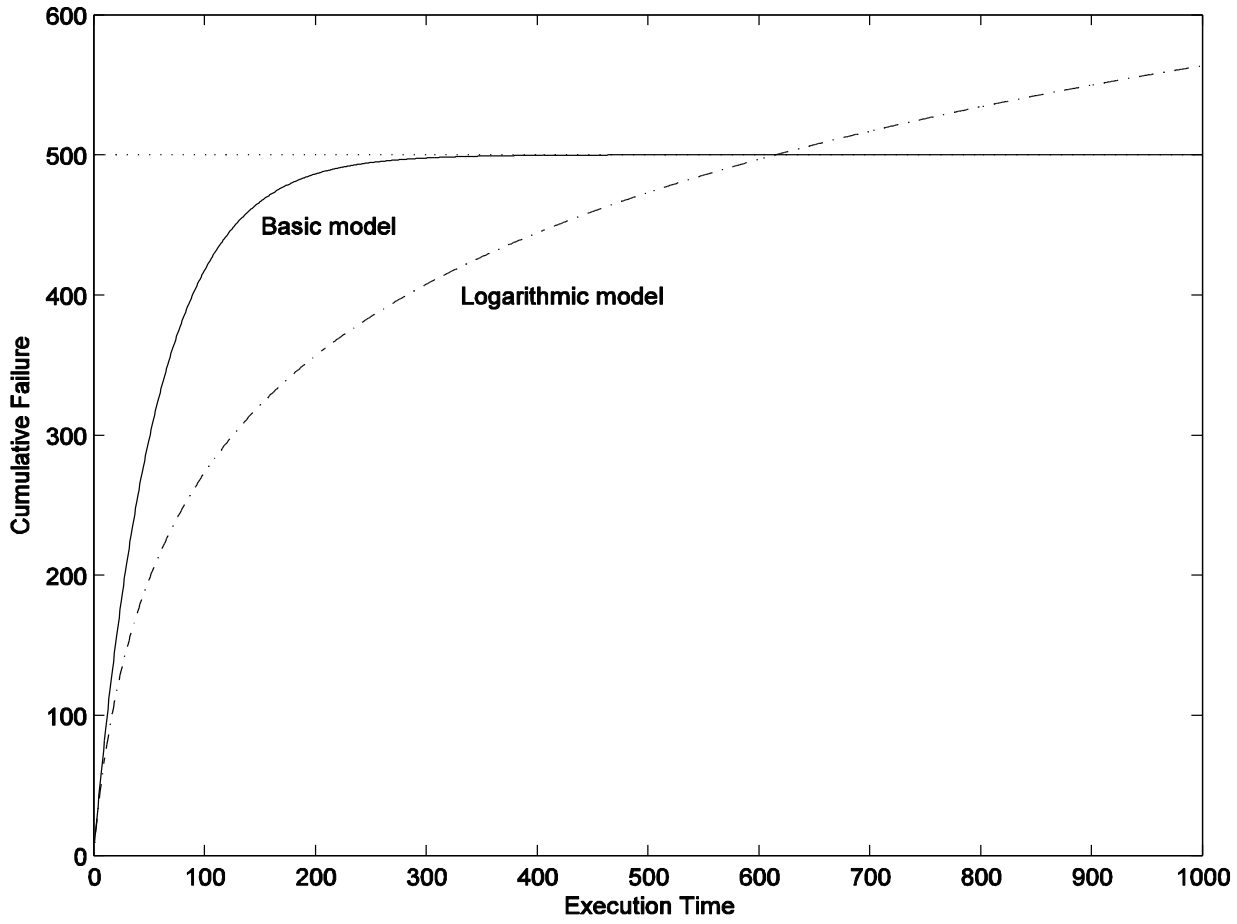


Figure 15.4: Cumulative failure μ as a function of execution time τ (λ_0 = 9 failures/unit time, v_0 = 500 failures, θ = 0.0075).

© Naik & Tripathy

Reliability Models

- Example

  Assume that a software system is undergoing system level testing. The initial failure intensity of the system was 25 failures/CPU hours, and the current failure intensity is 5 failures/CPU hour. It has been decided by the project manager that the system will be released only after the system reaches a reliability level of at most 0.001 failures/CPU hour. From their experience the management team estimates that the system will experience a total of 1200 failures over infinite time. Calculate the additional length of system testing required before the system can be released.

  - The system will experience a total of 1200 failures over infinite time. Thus, we use the Basic model.
  - λ_c and λ_r are the current failure intensity and the failure intensity at the time of release.
  - Assume that the current failure intensity has been achieved after executing the system for τ_c hours.
  - Let λ_r be achieved after testing the system for a total of τ_r hours.

Reliability Models

- (Example continued)
  - (τ_r - τ_c) denotes the additional execution time requires to achieve λ_r.

We can write λ_c and λ_r as follows.

$\lambda_c = \lambda_0.e^{-\lambda 0\ \tau c/\nu 0}$

$\lambda_r = \lambda_0.e^{-\lambda 0\ \tau r/\nu 0}$

$\lambda_c / \lambda_r = (\lambda_0.e^{-\lambda 0\ \tau c/\nu 0})/(\lambda_0.e^{-\lambda 0\ \tau r/\nu 0})$

$= e^{(\tau r - \tau c)\ \lambda 0/\nu 0}$

$\ln(\lambda_c / \lambda_r) = (\tau_r - \tau_c)\ \lambda_0/\nu_0$

$(\tau_r - \tau_c) = (\nu_0/ \lambda_0)\ln(\lambda_c / \lambda_r)$

$= (1200/25)\ln(5/0.001)$

$= 408.825$ hours

It is required to test the system for more time so that the CPU runs for another 408.825 hours to achieve the reliability level of 0.001 failures/hour.

Summary

- Reliability is a user-oriented quality factor relating to system operation.
- The chapter introduced the following.
  - Fault and failure
  - Execution and calendar time
  - Time interval between failures
  - Failures in periodic intervals
  - Failure intensity
- Software reliability was defined in two ways.
  - The probability of failure-free operation of a system for a specified time in a given environment.
  - Failure intensity is a measure of reliability.
- User's perception of reliability:
  - The number of faults in a system.
  - How a user operates a system.
- The number of faults in a system is influenced by the following:
  - Size and complexity of code.
  - Development process.
  - Personnel quality.
  - Operational environment
- Operational profile
  - A quantitative characterization of how actual users operate a system.
  - Tabular and graphical representation
- Applications of reliability metric
- Reliability models
  - Six assumptions
  - Two models
    - Basic
    - Logarithmic

© Naik & Tripathy